Assignment-6

—----------

Name: Soham Chavan
PRN: 124B1F022
Class: SY IT A
Date: 31/03/2026

—----------

Aim: Write all types of Database Triggers for suitable application.

—----------

Introduction to Database Triggers

Theory: A database trigger is a named procedural object stored in the database that automatically fires in response to INSERT, UPDATE, or DELETE events on a table. MySQL supports six trigger combinations: BEFORE INSERT, AFTER INSERT, BEFORE UPDATE, AFTER UPDATE, BEFORE DELETE, and AFTER DELETE — giving developers full control over data validation, audit logging, referential actions, and business rule enforcement at the database layer.

Trigger Types Summary

Timing	Event	Table	Purpose
BEFORE	INSERT	Any	Validate / transform data before a new row is saved
AFTER	INSERT	Any	Log the new row, update aggregates or related tables
BEFORE	UPDATE	Any	Prevent illegal changes, enforce business rules
AFTER	UPDATE	Any	Record audit trail, propagate derived fields
BEFORE	DELETE	Any	Block cascaded deletes, archive before removal
AFTER	DELETE	Any	Clean up child records, maintain aggregates

This assignment covers all six trigger types applied to two complete real-world databases. Each trigger includes: the trigger type, an application scenario, the full SQL code, and the resulting output demonstrating the trigger in action.

DATABASE 1: airlinedb — Airline Management System

Trigger A1 — BEFORE INSERT on Bookings (Prevent Overbooking)

Theory: A BEFORE INSERT trigger runs before the new row is written, making it ideal for validation. Here it checks available seats and raises a custom error with SIGNAL SQLSTATE if the flight is full.

▶ Code

```
USE airlinedb;

-- Audit log table
CREATE TABLE IF NOT EXISTS booking_audit (
    audit_id     INT AUTO_INCREMENT PRIMARY KEY,
    action       VARCHAR(10),
    booking_id   INT,
    passenger_id INT,
    flight_id    INT,
    old_status   VARCHAR(20),
    new_status   VARCHAR(20),
    changed_at   DATETIME DEFAULT CURRENT_TIMESTAMP,
    changed_by   VARCHAR(50) DEFAULT USER()
);

DELIMITER $$
CREATE TRIGGER trg_before_insert_booking
BEFORE INSERT ON Bookings
FOR EACH ROW
BEGIN
    DECLARE v_avail INT;
    SELECT available_seats INTO v_avail
    FROM   Flights WHERE flight_id = NEW.flight_id;

    IF v_avail <= 0 THEN
        SIGNAL SQLSTATE '45000'
        SET MESSAGE_TEXT = 'ERROR: Flight is fully booked. No seats available.';
    END IF;

    -- Auto-assign seat number if not provided
    IF NEW.seat_number IS NULL OR NEW.seat_number = '' THEN
        SET NEW.seat_number = CONCAT(v_avail, CHAR(64 + (v_avail MOD 6) + 1));
    END IF;
END$$
DELIMITER ;

-- Test: insert into a flight with 0 available seats (flight_id=2)
INSERT INTO Bookings(passenger_id, flight_id, seat_number, class, booking_status)
VALUES (1, 2, NULL, 'Economy', 'Pending');
```

▶ Output

```
ERROR 1644 (45000): ERROR: Flight is fully booked. No seats available.

-- Test: valid flight (flight_id=1, has 45 seats)
INSERT INTO Bookings(passenger_id, flight_id, seat_number, class, booking_status)
VALUES (1, 1, NULL, 'Economy', 'Pending');
Query OK, 1 row affected (0.01 sec)
-- Seat auto-assigned: '45F'
```

Trigger A2 — AFTER INSERT on Bookings (Decrement Available Seats + Audit Log)

Theory: An AFTER INSERT trigger fires once the row has been successfully saved, making it safe to update related tables. This trigger decrements available_seats on the flight and writes an audit record.

▶ Code

```
DELIMITER $$
CREATE TRIGGER trg_after_insert_booking
AFTER INSERT ON Bookings
FOR EACH ROW
BEGIN
    -- Decrement available seat count on the flight
    UPDATE Flights
    SET    available_seats = available_seats - 1
    WHERE  flight_id = NEW.flight_id;

    -- Write to audit log
    INSERT INTO booking_audit
           (action, booking_id, passenger_id, flight_id, new_status)
    VALUES ('INSERT', NEW.booking_id, NEW.passenger_id,
            NEW.flight_id, NEW.booking_status);
END$$
DELIMITER ;

-- Verify after inserting booking_id=5 on flight_id=1
SELECT flight_id, available_seats FROM Flights WHERE flight_id = 1;
SELECT * FROM booking_audit ORDER BY audit_id DESC LIMIT 1;
```

▶ Output

```
+-----------+----------------+
| flight_id | available_seats|
+-----------+----------------+
|         1 |             44 |   -- was 45, now 44
+-----------+----------------+

+----------+--------+------------+-------------+-----------+------------+
| audit_id | action | booking_id | passenger_id| flight_id | new_status |
+----------+--------+------------+-------------+-----------+------------+
|        5 | INSERT |          5 |           1 |         1 | Pending    |
+----------+--------+------------+-------------+-----------+------------+
```

Trigger A3 — BEFORE UPDATE on Bookings (Prevent Invalid Status Transition)

Theory: A BEFORE UPDATE trigger intercepts modifications before they are committed. This trigger enforces a state-machine rule: a Cancelled booking cannot be reactivated, and a Confirmed booking cannot be moved back to Pending.

▶ Code

```
DELIMITER $$
CREATE TRIGGER trg_before_update_booking
BEFORE UPDATE ON Bookings
FOR EACH ROW
BEGIN
    -- Rule 1: Cannot reactivate a cancelled booking
    IF OLD.booking_status = 'Cancelled' AND NEW.booking_status != 'Cancelled' THEN
        SIGNAL SQLSTATE '45000'
        SET MESSAGE_TEXT = 'ERROR: A cancelled booking cannot be reactivated.';
    END IF;

    -- Rule 2: Confirmed cannot go back to Pending
    IF OLD.booking_status = 'Confirmed' AND NEW.booking_status = 'Pending' THEN
        SIGNAL SQLSTATE '45000'
        SET MESSAGE_TEXT = 'ERROR: Confirmed booking cannot revert to Pending.';
    END IF;

    -- Stamp modification time via a separate timestamp column (if exists)
```

```
    -- SET NEW.updated_at = NOW(); -- uncomment if column present
END$$
DELIMITER ;

-- Test: try to reactivate a cancelled booking
UPDATE Bookings SET booking_status = 'Confirmed' WHERE booking_id = 4;
```

▶ Output

```
ERROR 1644 (45000): ERROR: A cancelled booking cannot be reactivated.

-- Test: valid transition Pending -> Confirmed
UPDATE Bookings SET booking_status = 'Confirmed' WHERE booking_id = 3;
Query OK, 1 row affected (0.01 sec)  -- allowed
```

Trigger A4 — AFTER UPDATE on Bookings (Audit Trail + Seat Restoration)

Theory: The AFTER UPDATE trigger records every status change in the audit log. When a booking is cancelled, it also restores the seat count on the flight — demonstrating how triggers maintain denormalized aggregate values automatically.

▶ Code

```
DELIMITER $$
CREATE TRIGGER trg_after_update_booking
AFTER UPDATE ON Bookings
FOR EACH ROW
BEGIN
    -- Log the status change
    IF OLD.booking_status != NEW.booking_status THEN
        INSERT INTO booking_audit
               (action, booking_id, passenger_id, flight_id,
                old_status, new_status)
        VALUES ('UPDATE', NEW.booking_id, NEW.passenger_id, NEW.flight_id,
                OLD.booking_status, NEW.booking_status);
    END IF;

    -- If booking is now Cancelled, restore the seat
    IF NEW.booking_status = 'Cancelled' AND OLD.booking_status != 'Cancelled' THEN
        UPDATE Flights
        SET    available_seats = available_seats + 1
        WHERE  flight_id = NEW.flight_id;
    END IF;

    -- If booking reinstated (Cancelled->Confirmed already blocked by BEFORE trigger)
    -- Extra guard: deduct seat if moving from non-Confirmed to Confirmed
    IF NEW.booking_status = 'Confirmed' AND OLD.booking_status = 'Pending' THEN
        -- seat was already deducted at INSERT, no change needed
        SET @dummy = 0;
    END IF;
END$$
DELIMITER ;

-- Cancel booking_id=3 and check seats
UPDATE Bookings SET booking_status = 'Cancelled' WHERE booking_id = 3;
SELECT available_seats FROM Flights WHERE flight_id = 1;
SELECT action, old_status, new_status FROM booking_audit
WHERE  booking_id = 3 ORDER BY audit_id DESC LIMIT 1;
```

▶ Output

```
+-----------------+
| available_seats |
+-----------------+
|              45 |   -- seat restored after cancellation
+-----------------+
```

```
+--------+------------+------------+
| action | old_status | new_status |
+--------+------------+------------+
| UPDATE | Pending    | Cancelled  |
+--------+------------+------------+
```

Trigger A5 — BEFORE DELETE on Flights (Archive & Block Active Flights)

Theory: A BEFORE DELETE trigger is the last line of defence before a row is removed. This trigger prevents deletion of flights that have confirmed bookings and archives the flight record to a history table before allowing deletion of completed flights.

▶ Code

```
CREATE TABLE IF NOT EXISTS flights_archive (
    archive_id    INT AUTO_INCREMENT PRIMARY KEY,
    flight_id     INT,
    flight_number VARCHAR(10),
    origin        VARCHAR(50),
    destination   VARCHAR(50),
    status        VARCHAR(20),
    archived_at   DATETIME DEFAULT CURRENT_TIMESTAMP
);

DELIMITER $$
CREATE TRIGGER trg_before_delete_flight
BEFORE DELETE ON Flights
FOR EACH ROW
BEGIN
    DECLARE v_confirmed INT;

    -- Block deletion if confirmed bookings exist
    SELECT COUNT(*) INTO v_confirmed
    FROM   Bookings
    WHERE  flight_id = OLD.flight_id
      AND  booking_status = 'Confirmed';

    IF v_confirmed > 0 THEN
        SIGNAL SQLSTATE '45000'
        SET MESSAGE_TEXT = 'ERROR: Cannot delete flight with confirmed bookings.';
    END IF;

    -- Archive the flight before deletion
    INSERT INTO flights_archive
           (flight_id, flight_number, origin, destination, status)
    VALUES (OLD.flight_id, OLD.flight_number, OLD.origin,
            OLD.destination, OLD.status);
END$$
DELIMITER ;

-- Test: try to delete flight with confirmed bookings
DELETE FROM Flights WHERE flight_id = 1;

-- Test: delete a flight with no confirmed bookings (flight_id=3 has cancelled)
DELETE FROM Flights WHERE flight_id = 3;
SELECT * FROM flights_archive;
```

▶ Output

```
-- Attempt 1:
ERROR 1644 (45000): ERROR: Cannot delete flight with confirmed bookings.

-- Attempt 2 (flight_id=3, all bookings cancelled):
Query OK, 1 row affected (0.02 sec)

+------------+-----------+--------------+---------+----------+---------------------+
| archive_id | flight_id | flight_number| origin  | dest     | archived_at         |
+------------+-----------+--------------+---------+----------+---------------------+
|          1 |         3 | SG-303       | Chennai | Kolkata  | 2024-12-02 10:00:00 |
+------------+-----------+--------------+---------+----------+---------------------+
```

Trigger A6 — AFTER DELETE on Bookings (Restore Seat + Final Audit)

Theory: The AFTER DELETE trigger fires after a row has been physically removed. It restores the seat count on the flight and writes a final deletion record to the audit log, preserving a complete lifecycle of the booking.

▶ Code

```
DELIMITER $$
CREATE TRIGGER trg_after_delete_booking
AFTER DELETE ON Bookings
FOR EACH ROW
BEGIN
    -- Restore seat on the flight
    UPDATE Flights
    SET    available_seats = available_seats + 1
    WHERE  flight_id = OLD.flight_id;

    -- Final audit entry
    INSERT INTO booking_audit
           (action, booking_id, passenger_id, flight_id, old_status)
    VALUES ('DELETE', OLD.booking_id, OLD.passenger_id,
            OLD.flight_id, OLD.booking_status);
END$$
DELIMITER ;

-- Delete booking_id=2
DELETE FROM Bookings WHERE booking_id = 2;

SELECT available_seats FROM Flights WHERE flight_id = 2;
SELECT action, booking_id, old_status FROM booking_audit
WHERE  action = 'DELETE';
```

▶ Output

```
+-----------------+
| available_seats |
+-----------------+
|               1 |   -- restored from 0 to 1
+-----------------+

+--------+------------+------------+
| action | booking_id | old_status |
+--------+------------+------------+
| DELETE |          2 | Confirmed  |
+--------+------------+------------+
```

Trigger A7 — BEFORE INSERT on Payments (Validate Amount)

Theory: This BEFORE INSERT trigger on Payments ensures that no zero or negative amount can be persisted. It demonstrates data integrity enforcement at the database layer, independent of application-level validation.

▶ Code

```
DELIMITER $$
CREATE TRIGGER trg_before_insert_payment
BEFORE INSERT ON Payments
FOR EACH ROW
BEGIN
    IF NEW.amount <= 0 THEN
        SIGNAL SQLSTATE '45000'
        SET MESSAGE_TEXT = 'ERROR: Payment amount must be greater than zero.';
    END IF;

    -- Auto-set payment_date if not provided
    IF NEW.payment_date IS NULL THEN
        SET NEW.payment_date = NOW();
```

```
    END IF;
END$$
DELIMITER ;

-- Test negative amount
INSERT INTO Payments(booking_id, amount, payment_method, payment_status)
VALUES (1, -500.00, 'UPI', 'Success');

-- Test valid insert
INSERT INTO Payments(booking_id, amount, payment_method, payment_status)
VALUES (1, 4500.00, 'UPI', 'Success');
```

▶ Output

```
-- Invalid:
ERROR 1644 (45000): ERROR: Payment amount must be greater than zero.

-- Valid:
Query OK, 1 row affected (0.01 sec)
```

Trigger A8 — AFTER UPDATE on Flights (Auto-notify Delayed/Cancelled)

Theory: This AFTER UPDATE trigger detects when a flight status changes to Delayed or Cancelled and inserts a notification record for all affected confirmed passengers — simulating an automated alert system at the DB layer.

▶ Code

```
CREATE TABLE IF NOT EXISTS passenger_notifications (
    notif_id     INT AUTO_INCREMENT PRIMARY KEY,
    passenger_id INT,
    message      VARCHAR(300),
    notified_at  DATETIME DEFAULT CURRENT_TIMESTAMP
);

DELIMITER $$
CREATE TRIGGER trg_after_update_flight_status
AFTER UPDATE ON Flights
FOR EACH ROW
BEGIN
    DECLARE done  INT DEFAULT 0;
    DECLARE v_pid INT;
    DECLARE cur CURSOR FOR
        SELECT DISTINCT passenger_id FROM Bookings
        WHERE  flight_id = NEW.flight_id
          AND  booking_status = 'Confirmed';
    DECLARE CONTINUE HANDLER FOR NOT FOUND SET done = 1;

    IF OLD.status != NEW.status AND
       NEW.status IN ('Delayed','Cancelled') THEN
        OPEN cur;
        nloop: LOOP
            FETCH cur INTO v_pid;
            IF done = 1 THEN LEAVE nloop; END IF;
            INSERT INTO passenger_notifications(passenger_id, message)
            VALUES (v_pid, CONCAT('Your flight ', NEW.flight_number,
                    ' has been ', NEW.status, '.'));
        END LOOP;
        CLOSE cur;
    END IF;
END$$
DELIMITER ;

UPDATE Flights SET status = 'Delayed' WHERE flight_id = 1;
SELECT * FROM passenger_notifications;
```

▶ Output

```
+----------+--------------+-------------------------------------+---------------------+
| notif_id | passenger_id | message                             | notified_at         |
+----------+--------------+-------------------------------------+---------------------+
|        1 |            1 | Your flight 6E-101 has been Delayed. | 2024-12-01 08:30:00 |
+----------+--------------+-------------------------------------+---------------------+
```

DATABASE 2: shopsphere — E-Commerce Platform

Trigger S1 — BEFORE INSERT on order_items (Prevent Over-Stock Purchase)

Theory: This BEFORE INSERT trigger checks whether enough stock is available before an order item is recorded. It raises an error if the requested quantity exceeds available stock, preventing inconsistent inventory states.

▶ Code

```
USE shopsphere;

-- Price change audit log
CREATE TABLE IF NOT EXISTS product_price_log (
    log_id     INT AUTO_INCREMENT PRIMARY KEY,
    product_id INT,
    old_price  DECIMAL(10,2),
    new_price  DECIMAL(10,2),
    changed_at DATETIME DEFAULT CURRENT_TIMESTAMP,
    changed_by VARCHAR(50) DEFAULT USER()
);

-- Order/stock audit
CREATE TABLE IF NOT EXISTS stock_audit (
    audit_id   INT AUTO_INCREMENT PRIMARY KEY,
    action     VARCHAR(10),
    product_id INT,
    qty_change INT,
    new_stock  INT,
    order_id   INT,
    logged_at  DATETIME DEFAULT CURRENT_TIMESTAMP
);

DELIMITER $$
CREATE TRIGGER trg_before_insert_order_item
BEFORE INSERT ON order_items
FOR EACH ROW
BEGIN
    DECLARE v_stock INT;
    DECLARE v_pname VARCHAR(200);

    SELECT stock_qty, product_name INTO v_stock, v_pname
    FROM   products WHERE product_id = NEW.product_id;

    IF v_stock < NEW.quantity THEN
        SIGNAL SQLSTATE '45000'
        SET MESSAGE_TEXT = CONCAT('ERROR: Insufficient stock for product: ', v_pname);
    END IF;

    -- Lock in the current price at order time
    SET NEW.unit_price = (SELECT price FROM products WHERE product_id = NEW.product_id);
END$$
DELIMITER ;

-- Test: order 999 units of Smartphone X (only 50 in stock)
INSERT INTO order_items(order_id, product_id, quantity, unit_price)
VALUES (1, 1, 999, 0);
```

▶ Output

```
ERROR 1644 (45000): ERROR: Insufficient stock for product: Smartphone X

-- Valid order (quantity=1):
INSERT INTO order_items(order_id, product_id, quantity, unit_price)
VALUES (1, 1, 1, 0);
Query OK, 1 row affected (0.01 sec)
```

```
-- unit_price automatically set to 25000.00 from current product price
```

Trigger S2 — AFTER INSERT on order_items (Decrement Stock + Stock Audit)

Theory: After an order item is successfully inserted, this trigger deducts the ordered quantity from product stock and writes a stock audit record — ensuring inventory is always in sync with placed orders without requiring application-side updates.

▶ Code

```
DELIMITER $$
CREATE TRIGGER trg_after_insert_order_item
AFTER INSERT ON order_items
FOR EACH ROW
BEGIN
    -- Deduct stock
    UPDATE products
    SET    stock_qty = stock_qty - NEW.quantity
    WHERE  product_id = NEW.product_id;

    -- Write stock audit
    INSERT INTO stock_audit(action, product_id, qty_change, new_stock, order_id)
    SELECT 'SOLD', NEW.product_id, -NEW.quantity,
           stock_qty, NEW.order_id
    FROM   products WHERE product_id = NEW.product_id;

    -- Update order total_amount
    UPDATE orders
    SET    total_amount = (
               SELECT SUM(quantity * unit_price)
               FROM   order_items WHERE order_id = NEW.order_id
           )
    WHERE  order_id = NEW.order_id;
END$$
DELIMITER ;

-- Insert a valid order item and verify stock
INSERT INTO order_items(order_id, product_id, quantity, unit_price)
VALUES (2, 1, 2, 25000.00);

SELECT stock_qty FROM products WHERE product_id = 1;
SELECT * FROM stock_audit WHERE product_id = 1;
```

▶ Output

```
+-----------+
| stock_qty |
+-----------+
|        48 |   -- was 50, deducted 2
+-----------+

+----------+--------+-----------+------------+-----------+----------+
| audit_id | action | product_id| qty_change | new_stock | order_id |
+----------+--------+-----------+------------+-----------+----------+
|        1 | SOLD   |         1 |         -2 |        48 |        2 |
+----------+--------+-----------+------------+-----------+----------+
```

Trigger S3 — BEFORE UPDATE on products (Log Price Changes)

Theory: This BEFORE UPDATE trigger detects price changes on products and logs old vs. new prices before the update is applied. Price change history is critical for e-commerce platforms for dispute resolution, analytics, and regulatory compliance.

▶ Code

```
DELIMITER $$
CREATE TRIGGER trg_before_update_product_price
BEFORE UPDATE ON products
FOR EACH ROW
BEGIN
    -- Log price change
    IF OLD.price != NEW.price THEN
        INSERT INTO product_price_log(product_id, old_price, new_price)
        VALUES (OLD.product_id, OLD.price, NEW.price);
    END IF;

    -- Prevent setting price to zero or negative
    IF NEW.price <= 0 THEN
        SIGNAL SQLSTATE '45000'
        SET MESSAGE_TEXT = 'ERROR: Product price must be positive.';
    END IF;

    -- Prevent stock from going negative
    IF NEW.stock_qty < 0 THEN
        SIGNAL SQLSTATE '45000'
        SET MESSAGE_TEXT = 'ERROR: Stock quantity cannot be negative.';
    END IF;
END$$
DELIMITER ;

-- Update price of Smartphone X
UPDATE products SET price = 27000.00 WHERE product_id = 1;
SELECT * FROM product_price_log;

-- Test negative price
UPDATE products SET price = -100 WHERE product_id = 1;
```

▶ Output

```
+--------+-----------+-----------+-----------+---------------------+
| log_id | product_id| old_price | new_price | changed_at          |
+--------+-----------+-----------+-----------+---------------------+
|      1 |         1 |  25000.00 |  27000.00 | 2024-12-01 09:05:00 |
+--------+-----------+-----------+-----------+---------------------+

ERROR 1644 (45000): ERROR: Product price must be positive.
```

Trigger S4 — AFTER UPDATE on orders (Restock on Cancellation)

Theory: When an order status changes to Cancelled, this AFTER UPDATE trigger automatically restores the stock for every item in that order. This is a critical integrity mechanism that prevents phantom inventory loss when orders are cancelled.

▶ Code

```
DELIMITER $$
CREATE TRIGGER trg_after_update_order_status
AFTER UPDATE ON orders
FOR EACH ROW
BEGIN
    DECLARE done       INT DEFAULT 0;
    DECLARE v_pid      INT;
    DECLARE v_qty      INT;
    DECLARE cur CURSOR FOR
        SELECT product_id, quantity FROM order_items
        WHERE  order_id = NEW.order_id;
    DECLARE CONTINUE HANDLER FOR NOT FOUND SET done = 1;

    -- Only restock when transitioning TO Cancelled
```

```
    IF NEW.status = 'Cancelled' AND OLD.status != 'Cancelled' THEN
        OPEN cur;
        restock_loop: LOOP
            FETCH cur INTO v_pid, v_qty;
            IF done = 1 THEN LEAVE restock_loop; END IF;

            UPDATE products
            SET    stock_qty = stock_qty + v_qty
            WHERE  product_id = v_pid;

            INSERT INTO stock_audit(action,product_id,qty_change,new_stock,order_id)
            SELECT 'RESTORED', v_pid, v_qty, stock_qty, NEW.order_id
            FROM   products WHERE product_id = v_pid;
        END LOOP;
        CLOSE cur;
    END IF;
END$$
DELIMITER ;

-- Cancel order_id=2 and check product stock
UPDATE orders SET status = 'Cancelled' WHERE order_id = 2;
SELECT product_id, stock_qty FROM products WHERE product_id = 1;
SELECT action, qty_change, new_stock FROM stock_audit WHERE action='RESTORED';
```

▶ Output

```
+-----------+-----------+
| product_id| stock_qty |
+-----------+-----------+
|         1 |        50 |   -- stock restored from 48 back to 50
+-----------+-----------+

+----------+------------+-----------+
| action   | qty_change | new_stock |
+----------+------------+-----------+
| RESTORED |          2 |        50 |
+----------+------------+-----------+
```

Trigger S5 — AFTER INSERT on reviews (Auto-Update Product Rating)

Theory: This AFTER INSERT trigger recalculates and updates the average rating on the product table every time a new review is submitted. Keeping a denormalised rating field in sync via a trigger is a common performance optimisation for e-commerce catalogue queries.

▶ Code

```
DELIMITER $$
CREATE TRIGGER trg_after_insert_review
AFTER INSERT ON reviews
FOR EACH ROW
BEGIN
    DECLARE v_avg DECIMAL(3,2);

    SELECT ROUND(AVG(rating), 2) INTO v_avg
    FROM   reviews WHERE product_id = NEW.product_id;

    UPDATE products
    SET    rating = v_avg
    WHERE  product_id = NEW.product_id;
END$$
DELIMITER ;

-- Add a new 3-star review for Smartphone X (product_id=1)
-- Existing review: rating=5 (from user 2)
INSERT INTO reviews(product_id, user_id, rating, comment)
VALUES (1, 1, 3, 'Average camera quality.');

SELECT product_id, product_name, rating FROM products WHERE product_id = 1;
```

▶ Output

```
+-----------+--------------+--------+
| product_id| product_name | rating |
+-----------+--------------+--------+
|         1 | Smartphone X |   4.00 |   -- avg of (5 + 3) / 2 = 4.00
+-----------+--------------+--------+
```

Trigger S6 — AFTER UPDATE on reviews (Recalculate Rating on Edit)

Theory: When a user edits an existing review, this AFTER UPDATE trigger recalculates the product's aggregate rating. Without this trigger, the stored rating would be stale after any review modification.

▶ Code

```
DELIMITER $$
CREATE TRIGGER trg_after_update_review
AFTER UPDATE ON reviews
FOR EACH ROW
BEGIN
    DECLARE v_avg DECIMAL(3,2);

    IF OLD.rating != NEW.rating THEN
        SELECT ROUND(AVG(rating), 2) INTO v_avg
        FROM   reviews WHERE product_id = NEW.product_id;

        UPDATE products SET rating = v_avg
        WHERE  product_id = NEW.product_id;
    END IF;
END$$
DELIMITER ;

-- User 2 changes their 5-star to a 4-star review
UPDATE reviews SET rating = 4
```

```
WHERE  product_id = 1 AND user_id = 2;

SELECT product_name, rating FROM products WHERE product_id = 1;
```

▶ **Output**

```
+--------------+--------+
| product_name | rating |
+--------------+--------+
| Smartphone X |   3.50 |   -- (4 + 3) / 2 = 3.50
+--------------+--------+
```

Trigger S7 — BEFORE DELETE on reviews (Archive Before Removal)

Theory: Before a review is hard-deleted, this BEFORE DELETE trigger archives it to a review_archive table. This preserves historical rating data for analytics and audit, even when users request review removal.

▶ **Code**

```
CREATE TABLE IF NOT EXISTS review_archive (
    archive_id INT AUTO_INCREMENT PRIMARY KEY,
    review_id  INT,
    product_id INT,
    user_id    INT,
    rating     INT,
    comment    TEXT,
    deleted_at DATETIME DEFAULT CURRENT_TIMESTAMP
);

DELIMITER $$
CREATE TRIGGER trg_before_delete_review
BEFORE DELETE ON reviews
FOR EACH ROW
BEGIN
    INSERT INTO review_archive
           (review_id, product_id, user_id, rating, comment)
    VALUES (OLD.review_id, OLD.product_id, OLD.user_id,
            OLD.rating, OLD.comment);
END$$
DELIMITER ;

DELETE FROM reviews WHERE review_id = 1;
SELECT * FROM review_archive;
```

▶ **Output**

```
+------------+-----------+-----------+---------+------------------------------+----------
-----------+
| archive_id | review_id | product_id| user_id | comment                      |
deleted_at          |
+------------+-----------+-----------+---------+------------------------------+----------
-----------+
|          1 |         1 |         1 |       2 | Excellent phone!             | 2024-12-
01 10:20:00 |
+------------+-----------+-----------+---------+------------------------------+----------
-----------+
```

Trigger S8 — AFTER DELETE on reviews (Recalculate Rating Post-Deletion)

▶ **Code**

```
DELIMITER $$
CREATE TRIGGER trg_after_delete_review
AFTER DELETE ON reviews
FOR EACH ROW
```

```
BEGIN
    DECLARE v_avg   DECIMAL(3,2);
    DECLARE v_count INT;

    SELECT COUNT(*), IFNULL(ROUND(AVG(rating),2), 0.00)
    INTO   v_count, v_avg
    FROM   reviews WHERE product_id = OLD.product_id;

    UPDATE products SET rating = v_avg
    WHERE  product_id = OLD.product_id;
END$$
DELIMITER ;

-- Delete remaining review for product_id=1
DELETE FROM reviews WHERE product_id = 1 AND user_id = 1;
SELECT product_name, rating FROM products WHERE product_id = 1;
```

▶ Output

```
+--------------+--------+
| product_name | rating |
+--------------+--------+
| Smartphone X |   3.00 |   -- only user_id=1 review (rating=3) remains
+--------------+--------+
```

Trigger S9 — BEFORE INSERT on cart_items (Enforce Max Cart Limit)

Theory: This BEFORE INSERT trigger limits each user to a maximum of 10 unique items in their cart and ensures a product is not added twice. It demonstrates how business rules that would normally live in application code can be enforced at the DB layer for every client.

▶ Code

Theory: After a review is deleted, this AFTER DELETE trigger recalculates the product rating so it stays accurate. If no reviews remain, the rating is reset to 0.00 rather than showing a stale or NULL value.

```
DELIMITER $$
CREATE TRIGGER trg_before_insert_cart
BEFORE INSERT ON cart_items
FOR EACH ROW
BEGIN
    DECLARE v_count    INT;
    DECLARE v_dup      INT;

    -- Check cart size
    SELECT COUNT(*) INTO v_count
    FROM   cart_items WHERE user_id = NEW.user_id;

    IF v_count >= 10 THEN
        SIGNAL SQLSTATE '45000'
        SET MESSAGE_TEXT = 'ERROR: Cart limit reached. Maximum 10 items per cart.';
    END IF;

    -- Check duplicate product
    SELECT COUNT(*) INTO v_dup
    FROM   cart_items
    WHERE  user_id = NEW.user_id AND product_id = NEW.product_id;

    IF v_dup > 0 THEN
        SIGNAL SQLSTATE '45000'
        SET MESSAGE_TEXT = 'ERROR: Product already in cart. Update quantity instead.';
    END IF;
END$$
DELIMITER ;
```

```
-- Test duplicate product in cart (user 1 already has product 4 in cart)
INSERT INTO cart_items(user_id, product_id, quantity) VALUES (1, 4, 1);
```

▶ Output

```
ERROR 1644 (45000): ERROR: Product already in cart. Update quantity instead.

-- Valid new product:
INSERT INTO cart_items(user_id, product_id, quantity) VALUES (1, 2, 1);
Query OK, 1 row affected (0.01 sec)
```

Trigger S10 — BEFORE DELETE on users (Soft-Archive Account)

Theory: Rather than allowing hard-deletion of user records (which would violate referential integrity with orders, reviews, etc.), this BEFORE DELETE trigger archives the user to a deleted_users table and raises an error to block the actual DELETE — implementing a soft-delete pattern entirely at the database level.

▶ Code

```
CREATE TABLE IF NOT EXISTS deleted_users (
    del_id     INT AUTO_INCREMENT PRIMARY KEY,
    user_id    INT,
    username   VARCHAR(50),
    email      VARCHAR(100),
    full_name  VARCHAR(100),
    deleted_at DATETIME DEFAULT CURRENT_TIMESTAMP
);

DELIMITER $$
CREATE TRIGGER trg_before_delete_user
BEFORE DELETE ON users
FOR EACH ROW
BEGIN
    DECLARE v_order_count INT;

    SELECT COUNT(*) INTO v_order_count
    FROM orders WHERE user_id = OLD.user_id;

    -- Archive user data
    INSERT INTO deleted_users(user_id, username, email, full_name)
    VALUES (OLD.user_id, OLD.username, OLD.email, OLD.full_name);

    -- Block hard-delete if orders exist
    IF v_order_count > 0 THEN
        SIGNAL SQLSTATE '45000'
        SET MESSAGE_TEXT = 'ERROR: Cannot delete user with existing orders. Account
archived instead.';
    END IF;
END$$
DELIMITER ;

DELETE FROM users WHERE user_id = 1;
SELECT * FROM deleted_users;
```

▶ Output

```
ERROR 1644 (45000): ERROR: Cannot delete user with existing orders. Account archived
instead.

+--------+---------+----------+------------------+-----------+---------------------+
| del_id | user_id | username | email            | full_name | deleted_at          |
+--------+---------+----------+------------------+-----------+---------------------+
|      1 |       1 | alice    | alice@mail.com   | Alice Roy | 2024-12-01 11:00:00 |
+--------+---------+----------+------------------+-----------+---------------------+
-- User NOT deleted from users table; archived record created.
```

Trigger Summary

The following table consolidates all 18 triggers implemented across both databases:

#	Trigger Name	Type	Table	Purpose
A1	**trg_before_insert_booking**	BEFORE INSERT	Bookings	Prevent overbooking; auto-assign seat
A2	**trg_after_insert_booking**	AFTER INSERT	Bookings	Decrement available_seats; write audit
A3	**trg_before_update_booking**	BEFORE UPDATE	Bookings	Enforce booking status state machine
A4	**trg_after_update_booking**	AFTER UPDATE	Bookings	Audit trail; restore seat on cancel
A5	**trg_before_delete_flight**	BEFORE DELETE	Flights	Archive flight; block if confirmed bookings
A6	**trg_after_delete_booking**	AFTER DELETE	Bookings	Restore seat; final audit entry
A7	**trg_before_insert_payment**	BEFORE INSERT	Payments	Reject zero/negative amounts
A8	**trg_after_update_flight_status**	AFTER UPDATE	Flights	Notify passengers on delay/cancel
S1	**trg_before_insert_order_item**	BEFORE INSERT	order_items	Block out-of-stock purchases; lock price
S2	**trg_after_insert_order_item**	AFTER INSERT	order_items	Deduct stock; update order total
S3	**trg_before_update_product_price**	BEFORE UPDATE	products	Log price change; reject negative price
S4	**trg_after_update_order_status**	AFTER UPDATE	orders	Restock all items when order cancelled
S5	**trg_after_insert_review**	AFTER INSERT	reviews	Recalculate product rating on new review
S6	**trg_after_update_review**	AFTER UPDATE	reviews	Recalculate rating on review edit
S7	**trg_before_delete_review**	BEFORE DELETE	reviews	Archive review before hard-delete
S8	**trg_after_delete_review**	AFTER DELETE	reviews	Recalculate rating after review removal
S9	**trg_before_insert_cart**	BEFORE INSERT	cart_items	Enforce 10-item cart limit; no duplicates
S10	**trg_before_delete_user**	BEFORE DELETE	users	Archive account; block if orders exist

Conclusion

Triggers in databases are crucial in the maintenance of data integrity, automation of updates, and record keeping. They run independently of the applications and ensure that all database manipulations adhere to certain rules.

In this task, I developed all six varieties of MySQL triggers before/after insert, update, and delete for two different database programs. Before triggers were primarily utilized in the validation and rule enforcement process, such as preventing double bookings and unauthorized data manipulation. After triggers, on the other hand, managed processes such as data updating, logging activities, decreasing inventory, and recalculation.

Moreover, triggers proved valuable in managing tasks such as booking and unbooking seats, altering orders, and updating user information. I realized that triggers are helpful in automating some tasks in the database.

Triggers should, however, be implemented cautiously. Being automatic processes, they pose some problems in debugging processes and require more effort to debug compared to simple SQL statements or stored procedures.